claude-windows / df5d5fb2-534a-447a-8181-b222f94655f3

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\df5d5fb2-534a-447a-8181-b222f94655f3\subagents\workflows\wf_133acdee-dab\agent-a32b7af4bbe04e713.jsonl`
- SHA-256: `ba3a2c4000e44ea8e3a82b3a6c4e9bb9de6c26dd3d5c3425e13e85ce44f0ff16`
- Source modified: `2026-07-06T17:31:19+00:00`
- Imported at: `2026-07-08T15:59:35+00:00`
- project: `wf_133acdee-dab`
- session_id: `df5d5fb2-534a-447a-8181-b222f94655f3`

Transcript

1. user (2026-07-06T17:30:35.318Z)

```
You are an ADVERSARIAL VERIFIER. Try HARD to REFUTE the following review finding by
reading the ACTUAL code in C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer
(and the diff at C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-
highlight-indexer/df5d5fb2-534a-447a-8181-b222f94655f3/scratchpad/wf294_fix.diff). Default to
REFUTED if you cannot
concretely reproduce the defect by pointing at real code lines. A finding is CONFIRMED only if
you can trace a concrete input/state that produces the claimed wrong behavior in the code AS
WRITTEN (not as you imagine it). If the claimed line/behavior does not exist, or a guard/caller
already handles it, or it is out-of-scope speculation, mark REFUTED. If genuinely unclear after
reading, mark UNCERTAIN.

FINDING (from the conventions-tests lens):
severity=low | tests/test_worker.py:270
summary: No end-to-end worker test flows a REAL TrajectoryHybridSegmenter Failure through
cmd_infer; the rc-3 test uses a hand-rolled _FailSeg, and the segmenter unit tests stop at the
segmenter return boundary.
detail: test_cmd_infer_detector_failure_marks_report_and_marker_failed monkeypatches
segmenter_registry.get to return _FailSeg (test_worker.py line 204-215), a stand-in that just
returns a Failure. The real trajectory_hybrid Failure returns are only verified at the segmenter
level (test_tracknet_hybrid.py test_pipeline_healthcheck_failure_aborts /
test_pipeline_no_trajectory_aborts assert isinstance(res, Failure)). No test drives the ACTUAL
TrajectoryHybridSegmenter/WasbHybridSegmenter healthcheck-fail or no-trajectory path through
cmd_infer to confirm the two ends compose. Both halves are individually locked, so this is a
completeness gap rather than an uncovered hole, but a divergence between the real segmenter's
return type and cmd_infer's isinstance(result, Failure) unwrap (worker/__main__.py line 326-329)
would slip past both suites.
failure_scenario: If a future refactor changed TrajectoryHybridSegmenter to return, e.g., a
Success-wrapped Failure or a differently-typed sentinel on the abort paths, the segmenter tests
(which assert on the segmenter's own return) and the worker test (which injects _FailSeg) would
both still pass while cmd_infer silently mis-routed the real failure back to status="success".
suggested_fix: Add one worker test that registers/monkeypatches the real WasbHybridSegmenter
(with its runner/env mocked to force healthcheck-fail or run_predict->None, as
test_tracknet_hybrid already does) and asserts cmd_infer returns rc 3 with report.json
status="failed".

Verify against the real code and return your verdict.
```

2. assistant (2026-07-06T17:30:37.936Z)

I'll verify this finding by examining the actual code and test files.

3. user (2026-07-06T17:30:39.945Z)

```
190         assert len(rows) == 3  # header + 2 rallies
```

```
        assert json.loads(report.read_text())["segment_count"] == 2


    class _EmptySeg:
        """A segmenter that finds nothing ? a LEGITIMATE 0-rally match (a bare []).."""

        def __init__(self, db, config):
            pass

        def process_video(self, video_id, path, max_frames=None):
            return []


    class _FailSeg:
        """A segmenter whose DETECTOR failed/timed out ? returns a Failure (e.g. WASB
produced no
        trajectory), which is NOT the same as a 0-rally match. This is the wf294 shape."""

        def __init__(self, db, config):
            pass

        def process_video(self, video_id, path, max_frames=None):
            return Failure(
                message="WASB detector produced no trajectory (timed out or aborted)",
                is_recoverable=True,
            )


    def test_setup_logging_levels():
        import logging

        wmain._setup_logging(False)
        assert logging.getLogger().level == logging.INFO
        wmain._setup_logging(True)
        assert logging.getLogger().level == logging.DEBUG


    def test_cmd_infer_zero_segments_is_logged_loudly(tmp_path, monkeypatch, caplog):
        # The previous silent "0 segment(s)" gave nothing to analyse (the real cold-start
bug).
        monkeypatch.setattr(wmain, "compute_video_id", lambda p: "vidZ")
        monkeypatch.setattr(
            wmain.validator_registry,
            "run_all_with_context",
            lambda path, cfg: ([_OKResult()], {}),
        )
        monkeypatch.setattr(wmain.segmenter_registry, "get", lambda name: _EmptySeg)
        monkeypatch.setattr(wmain, "emit_indexer_report", lambda **k: None)
        monkeypatch.setenv("RALLY_DETECTOR_IMPL", "native")
        video = tmp_path / "in.mp4"
        video.write_bytes(b"FAKE")
        out = tmp_path / "bucket"

        # Call cmd_infer directly (not via main(), whose _setup_logging force-reconfigures
handlers and
        # would detach caplog's). The warning is emitted by cmd_infer regardless of who
configures logging.
        args = wmain.build_parser().parse_args(
            [
                "infer",
                "--video-uri",
                str(video),
                "--out-uri",
                str(out),
                "--scratch",
```

```
252                 str(tmp_path / "s"),
253                 "--segmenter",
254                 "wasb_hybrid",
255             ]
256         )
257         with caplog.at_level("WARNING", logger="backend.worker"):
258             rc = wmain.cmd_infer(args)
259         assert rc == 0  # still a clean run, just empty ? but now EXPLAINED
260         msgs = " ".join(r.getMessage() for r in caplog.records)
261         assert "0 segments" in msgs and "vidZ" in msgs
262         assert "detector_impl=native" in msgs  # surfaces the resolved detector
263         assert "native runner UNAVAILABLE" in msgs  # points at the likely cause
264         # A segmenter that returns a bare [] is a LEGITIMATE 0-rally match ? status stays
"success"
265         # (contrast test_cmd_infer_detector_failure_*, where a Failure surfaces as
status="failed").
266         report = json.loads((out / "outputs" / "vidZ" / "report.json").read_text())
267         assert report["segment_count"] == 0 and report["status"] == "success"
268
269
270     def test_cmd_infer_detector_failure_marks_report_and_marker_failed(
271         tmp_path, monkeypatch
272     ):
273         """wf294 fix: a segmenter that returns a Failure (detector timeout/abort) must NOT
be written
274         as a 0-segment "success". cmd_infer returns rc 3 and writes report.json + the done-
marker with
275         status="failed" so the control plane / alpha-user surface can show an error /
retry."""
276         monkeypatch.setattr(wmain, "compute_video_id", lambda p: "vidX")
277         monkeypatch.setattr(
278             wmain.validator_registry,
279             "run_all_with_context",
280             lambda path, cfg: ([_OKResult()], {}),
281         )
282         monkeypatch.setattr(wmain.segmenter_registry, "get", lambda name: _FailSeg)
283         monkeypatch.setattr(wmain, "emit_indexer_report", lambda **k: None)
284         video = tmp_path / "in.mp4"
285         video.write_bytes(b"FAKE")
286         out = tmp_path / "bucket"
287         done = tmp_path / "bucket" / "_dispatch" / "tok.done"
288
289         rc = wmain.main(
290             [
291                 "infer",
292                 "--video-uri",
293                 str(video),
294                 "--out-uri",
295                 str(out),
296                 "--scratch",
297                 str(tmp_path / "s"),
298                 "--segmenter",
299                 "wasb_hybrid",
300                 "--done-uri",
301                 str(done),
302             ]
303         )
304         # A detector failure is rc 3 (distinct from validation's rc 2 and a clean rc 0).
305         assert rc == 3
306         # report.json is written as an explicit FAILURE, not a false empty "success".
307         report = json.loads((out / "outputs" / "vidX" / "report.json").read_text())
308         assert report["status"] == "failed" and report["segment_count"] == 0
309         assert report["segments"] == []
```

4. user (2026-07-06T17:30:41.342Z)

```
diff --git a/backend/pipeline/segmenters/trajectory_hybrid.py
b/backend/pipeline/segmenters/trajectory_hybrid.py
index 503f6d8..f96b495 100644
--- a/backend/pipeline/segmenters/trajectory_hybrid.py
+++ b/backend/pipeline/segmenters/trajectory_hybrid.py
@@ -22,7 +22,7 @@ import concurrent.futures
 import logging
 import os
 import time
-from typing import Any, Dict, List, Optional, Tuple
+from typing import Any, Dict, List, Optional, Tuple, Union

 import cv2

@@ -30,6 +30,9 @@ from backend.config.models import IndexingConfig
 from backend.pipeline.detectors.base import DetectorRunner
 from backend.pipeline.segmenters.base import BasePlaySegmenter
 from backend.providers import provider_registry
+
+# Standardized Success|Failure contract: a detector Failure is NOT a 0-rally result.
+from backend.results import Failure
 from backend.sports import sport_registry
 from backend.utils.paths import output_base
 from backend.utils.video import extract_video_chunk, get_video_duration
@@ -174,10 +177,13 @@ class TrajectoryHybridSegmenter(BasePlaySegmenter):
         video_path: str,
         player_pool: Optional[List[str]] = None,
         max_frames: Optional[int] = None,
-    ) -> List[Dict[str, Any]]:
-        # override-ignore: the ABC declares the Result contract (Success|Failure)
-        # but every live segmenter still returns a bare list ? the documented
-        # deliberate-incremental gap (CODE_MAP gotchas; main.py handles both).
+    ) -> "Union[List[Dict[str, Any]], Failure]":
+        # Partial adoption of the Result contract (the ABC declares Success|Failure).
+        # A DETECTOR failure ? the env healthcheck not ready, or run_predict timing out
/
+        # aborting with no trajectory ? returns a ``Failure`` so it is NOT confused
with a
+        # genuine 0-rally video (which still returns a bare ``[]``). Every caller
+        # (worker.cmd_infer, main.py, orchestration) branches on ``isinstance(result,
Failure)``;
+        # the bare-list success/empty paths are the documented incremental gap
(CODE_MAP gotchas).
         label = self.display_label
         # --- Sport profile + AI provider wiring (identical across segmenters) ---
         sport_name = self.config.sport
@@ -267,7 +273,12 @@ class TrajectoryHybridSegmenter(BasePlaySegmenter):
         ok, msg = runner.healthcheck()
         if not ok:
             logger.error("%s detector environment not ready: %s", label, msg)
-            return []
+            # A detector FAILURE, not a 0-rally video ? surface it so the caller marks
the run
+            # failed/retryable instead of writing an empty-but-"successful" result.
+            return Failure(
+                message=f"{label} detector environment not ready: {msg}",
+                is_recoverable=True,
+            )
         logger.info("%s detector env OK: %s", label, msg)

         # --- Phase 2: trajectory -> candidate rally windows ---
@@ -278,7 +289,17 @@ class TrajectoryHybridSegmenter(BasePlaySegmenter):
         csv_path = runner.run_predict(video_path, out_dir, video_id=video_id)
         if not csv_path:
             logger.error("%s produced no trajectory; aborting.", label)
```

```
60      -            return []
61      +            # The cold-start / timeout failure mode (e.g. WASB native inference timed
out after
62      +            # timeout_sec, or the detector env yielded nothing). This is a DETECTOR
failure ? NOT
63      +            # a legitimately empty match ? so return a Failure the worker marks as
failed/retryable
64      +            # rather than a silent 0-segment "success" (the wf294 incident).
65      +            return Failure(
66      +                message=(
67      +                    f"{label} detector produced no trajectory (timed out or aborted) ?
"
68      +                    "detector failure, not a 0-rally video."
69      +                ),
70      +                is_recoverable=True,
71      +            )
72
73               points = runner.parse_trajectory_csv(csv_path)
74               logger.info(
75      diff --git a/backend/worker/__main__.py b/backend/worker/__main__.py
76      index 461d7ca..88c50f6 100644
77      --- a/backend/worker/__main__.py
78      +++ b/backend/worker/__main__.py
79      @@ -114,6 +114,42 @@ def _write_report_json(
80               )
81
82
83      +def _serialize_and_push_outputs(
84      +    scratch: str,
85      +    out_uri: str,
86      +    video_id: str,
87      +    segments: List[dict],
88      +    status: str,
89      +    storage_cfg: dict,
90      +    video_uri: str = "",
91      +) -> List[str]:
92      +    """Write intervals.csv + report.json under ``<scratch>/outputs/<video_id>/`` and
push both to
93      +    ``<out_uri>/outputs/<video_id>/``; return the pushed URIs.
94      +
95      +    Shared by the SUCCESS path and the failure path (``_fail``) so a detector
timeout/abort leaves a
96      +    ``status="failed"`` report.json in the bucket ? the control plane's source of truth
97      +    (``orchestration.cached_process_result`` / ``probe_process_cache`` treat any
non-``success``
98      +    report as "no usable result" ? retry, and the restart-recovery poll waits for
report.json to
99      +    EXIST) ? instead of a missing artifact or a false 0-segment ``"success"``."""
100     +    out_local = os.path.join(scratch, "outputs", video_id)
101     +    os.makedirs(out_local, exist_ok=True)
102     +    _write_intervals_csv(segments, os.path.join(out_local, "intervals.csv"))
103     +    _write_report_json(
104     +        video_id,
105     +        segments,
106     +        status,
107     +        os.path.join(out_local, "report.json"),
108     +        video_uri=video_uri,
109     +    )
110     +    out_prefix = out_uri.rstrip("/")
111     +    pushed: List[str] = []
112     +    for fn in sorted(os.listdir(out_local)):
113     +        uri = f"{out_prefix}/outputs/{video_id}/{fn}"
114     +        storage.put(os.path.join(out_local, fn), uri, config=storage_cfg)
115     +        pushed.append(uri)
116     +    return pushed
```

```
117     +
118     +
119      def _run_startup_checks(config: Any) -> bool:
120          """M5 per-profile startup checks for the worker (the compute process, so it probes
CUDA
121          best-effort). Logs warnings; returns False on a FATAL coherence error so the caller
aborts
122     @@ -237,9 +273,26 @@ def cmd_infer(args: argparse.Namespace) -> int:
123          )
124
125          def _fail(rc: int) -> int:
126     -        # M6: on a worker-side FAILURE, write the done-marker (the REAL rc, 0 segments)
so a remote
127     -        # dispatcher's bucket-poll terminates FAST + accurately instead of hanging
until its poll
128     -        # budget expires. DEFAULT-OFF: no --done-uri ? no marker written (the unchanged
local run).
129     +        # A worker-side FAILURE (validation, unknown segmenter, or a detector
timeout/abort the
130     +        # segmenter surfaced as a Failure) must NOT masquerade as a 0-rally "success".
Write + push
131     +        # a status="failed" report.json/intervals.csv (0 segments) so the control
plane's bucket
132     +        # truth (submit-cache probe, restart-recovery) and the alpha-user result
surface see an
133     +        # explicit FAILURE ? the wf294 silent-failure fix. Best-effort: a push hiccup
must never
134     +        # mask the original failure. THEN the M6 done-marker (real rc + status) so a
remote
135     +        # dispatcher's bucket-poll terminates FAST instead of hanging until its budget
expires.
136     +        try:
137     +            _serialize_and_push_outputs(
138     +                scratch,
139     +                args.out_uri,
140     +                video_id,
141     +                [],
142     +                "failed",
143     +                storage_cfg,
144     +                str(getattr(args, "video_uri", "") or ""),
145     +            )
146     +        except Exception as e:  # never mask the real failure on an artifact-push
hiccup
147     +            logger.warning("[worker] could not push failure artifacts: %s", e)
148     +        # DEFAULT-OFF: no --done-uri ? no marker written (the unchanged local run).
149              if getattr(args, "done_uri", None):
150                  _write_done_marker(
151                      args.done_uri, video_id, rc, 0, "failed", storage_cfg, scratch
152     @@ -313,24 +366,16 @@ def cmd_infer(args: argparse.Namespace) -> int:
153              )
154
155          # 5. SERIALIZE + 6. PUSH (outputs/<video_id>/?). idempotent on video_id.
156     -    out_local = os.path.join(scratch, "outputs", video_id)
157     -    os.makedirs(out_local, exist_ok=True)
158     -    _write_intervals_csv(segments, os.path.join(out_local, "intervals.csv"))
159     -    _write_report_json(
160     +    pushed = _serialize_and_push_outputs(
161     +        scratch,
162     +        args.out_uri,
163              video_id,
164              segments,
165              status,
166     -        os.path.join(out_local, "report.json"),
167     -        video_uri=str(getattr(args, "video_uri", "") or ""),
168     +        storage_cfg,
```

```
169     +        str(getattr(args, "video_uri", "") or ""),
170          )
171
172     -    out_prefix = args.out_uri.rstrip("/")
173     -    pushed = []
174     -    for fn in sorted(os.listdir(out_local)):
175     -        uri = f"{out_prefix}/outputs/{video_id}/{fn}"
176     -        storage.put(os.path.join(out_local, fn), uri, config=storage_cfg)
177     -        pushed.append(uri)
178     -
179          print(
180              f"[worker] infer done: {len(segments)} segment(s); pushed {len(pushed)}
artifact(s):"
181          )
182     diff --git a/tests/test_tracknet_hybrid.py b/tests/test_tracknet_hybrid.py
183     index 64909e7..c5ae442 100644
184     --- a/tests/test_tracknet_hybrid.py
185     +++ b/tests/test_tracknet_hybrid.py
186     @@ -5,6 +5,7 @@ healthcheck gating, candidate persistence, AI handoff, and DB
population/linking
187      from unittest.mock import MagicMock, patch
188
189      from backend.pipeline.segmenters.tracknet_hybrid import TrackNetHybridSegmenter
190     +from backend.results import Failure
191
192
193      def _window(start=1.0, end=4.0):
194     @@ -124,7 +125,10 @@ def test_pipeline_healthcheck_failure_aborts(
195
196          seg = TrackNetHybridSegmenter(MagicMock(), {"indexing": {}})
197          res = seg.process_video("v1", "clip.mp4")
198     -    assert res == []
199     +    # A detector-env failure is a Failure, NOT a 0-rally [] ? so the caller marks the
run failed
200     +    # instead of writing an empty "success".
201     +    assert isinstance(res, Failure)
202     +    assert "environment not ready" in res.message
203          # Should bail before ever running inference.
204          mock_runner_cls.return_value.run_predict.assert_not_called()
205
206     @@ -144,7 +148,36 @@ def test_pipeline_no_trajectory_aborts(
207          mock_provider_registry.get.return_value = MagicMock()
208
209          seg = TrackNetHybridSegmenter(MagicMock(), {"indexing": {}})
210     -    assert seg.process_video("v1", "clip.mp4") == []
211     +    res = seg.process_video("v1", "clip.mp4")
212     +    # A detector timeout/abort (run_predict yielded no trajectory) is a Failure, NOT a
0-rally [] ?
213     +    # this is exactly the wf294 silent-failure that the caller must now mark
failed/retryable.
214     +    assert isinstance(res, Failure)
215     +    assert "no trajectory" in res.message and res.is_recoverable
216     +
217     +
218     +@_patched
219     +def test_pipeline_zero_windows_is_legit_empty_not_failure(
220     +    mock_runner_cls,
221     +    mock_probe,
222     +    mock_dur,
223     +    mock_extract,
224     +    mock_provider_registry,
225     +    mock_env,
226     +):
227     +    """The other side of the wf294 distinction: the detector RAN fine (a trajectory was
produced +
```

```
    +    parsed) but yielded no candidate rally windows. That is a legitimately empty match
? a bare []
    +    (0-rally "success"), NOT a Failure."""
    +    mock_env.return_value = "dummy_key"
    +    mock_dur.return_value = 30.0
    +    mock_extract.return_value = True
    +    mock_runner_cls.return_value = _make_runner(windows=[])  # trajectory ok, zero
windows
    +    mock_provider_registry.get.return_value = MagicMock()
    +
    +    seg = TrackNetHybridSegmenter(MagicMock(), {"indexing": {}})
    +    res = seg.process_video("v1", "clip.mp4")
    +    assert res == [] and not isinstance(res, Failure)
    +    # The detector DID run (a genuine empty, not an abort).
    +    mock_runner_cls.return_value.run_predict.assert_called_once()


     @_patched
    diff --git a/tests/test_worker.py b/tests/test_worker.py
    index 96edbee..63cc18e 100644
    --- a/tests/test_worker.py
    +++ b/tests/test_worker.py
    @@ -10,6 +10,7 @@ import json
     import pytest

     from backend.config import load_config
    +from backend.results import Failure
     from backend.worker import __main__ as wmain

     # ------------------------------------------------------------------ pure helpers
    @@ -191,7 +192,7 @@ def test_cmd_infer_local_roundtrip(tmp_path, mocked_pipeline):


     class _EmptySeg:
    -    """A segmenter that finds nothing ? the cold-start failure mode."""
    +    """A segmenter that finds nothing ? a LEGITIMATE 0-rally match (a bare [])."""

         def __init__(self, db, config):
             pass
    @@ -200,6 +201,20 @@ class _EmptySeg:
             return []


    +class _FailSeg:
    +    """A segmenter whose DETECTOR failed/timed out ? returns a Failure (e.g. WASB
produced no
    +    trajectory), which is NOT the same as a 0-rally match. This is the wf294 shape."""
    +
    +    def __init__(self, db, config):
    +        pass
    +
    +    def process_video(self, video_id, path, max_frames=None):
    +        return Failure(
    +            message="WASB detector produced no trajectory (timed out or aborted)",
    +            is_recoverable=True,
    +        )
    +
    +
     def test_setup_logging_levels():
         import logging

    @@ -246,12 +261,56 @@ def test_cmd_infer_zero_segments_is_logged_loudly(tmp_path,
monkeypatch, caplog)
         assert "0 segments" in msgs and "vidZ" in msgs
         assert "detector_impl=native" in msgs  # surfaces the resolved detector
```

```
289          assert "native runner UNAVAILABLE" in msgs  # points at the likely cause
290      -    assert (
291      -        json.loads((out / "outputs" / "vidZ" / "report.json").read_text())[
292      -            "segment_count"
293      +    # A segmenter that returns a bare [] is a LEGITIMATE 0-rally match ? status stays
"success"
294      +    # (contrast test_cmd_infer_detector_failure_*, where a Failure surfaces as
status="failed").
295      +    report = json.loads((out / "outputs" / "vidZ" / "report.json").read_text())
296      +    assert report["segment_count"] == 0 and report["status"] == "success"
297      +
298      +
299      +def test_cmd_infer_detector_failure_marks_report_and_marker_failed(
300      +    tmp_path, monkeypatch
301      +):
302      +    """wf294 fix: a segmenter that returns a Failure (detector timeout/abort) must NOT
be written
303      +    as a 0-segment "success". cmd_infer returns rc 3 and writes report.json + the done-
marker with
304      +    status="failed" so the control plane / alpha-user surface can show an error /
retry."""
305      +    monkeypatch.setattr(wmain, "compute_video_id", lambda p: "vidX")
306      +    monkeypatch.setattr(
307      +        wmain.validator_registry,
308      +        "run_all_with_context",
309      +        lambda path, cfg: ([_OKResult()], {}),
310      +    )
311      +    monkeypatch.setattr(wmain.segmenter_registry, "get", lambda name: _FailSeg)
312      +    monkeypatch.setattr(wmain, "emit_indexer_report", lambda **k: None)
313      +    video = tmp_path / "in.mp4"
314      +    video.write_bytes(b"FAKE")
315      +    out = tmp_path / "bucket"
316      +    done = tmp_path / "bucket" / "_dispatch" / "tok.done"
317      +
318      +    rc = wmain.main(
319      +        [
320      +            "infer",
321      +            "--video-uri",
322      +            str(video),
323      +            "--out-uri",
324      +            str(out),
325      +            "--scratch",
326      +            str(tmp_path / "s"),
327      +            "--segmenter",
328      +            "wasb_hybrid",
329      +            "--done-uri",
330      +            str(done),
331               ]
332      -        == 0
333          )
334      +    # A detector failure is rc 3 (distinct from validation's rc 2 and a clean rc 0).
335      +    assert rc == 3
336      +    # report.json is written as an explicit FAILURE, not a false empty "success".
337      +    report = json.loads((out / "outputs" / "vidX" / "report.json").read_text())
338      +    assert report["status"] == "failed" and report["segment_count"] == 0
339      +    assert report["segments"] == []
340      +    # ...and the completion marker the cloud dispatcher bucket-polls carries the same
verdict.
341      +    marker = json.loads(done.read_text())
342      +    assert marker["returncode"] == 3 and marker["status"] == "failed"
343      +    assert marker["segment_count"] == 0 and marker["video_id"] == "vidX"
344
345
346       def test_cmd_infer_validation_failure_returns_2(tmp_path, monkeypatch):
347
```

5. assistant (2026-07-06T17:30:47.306Z)

The finding is a completeness gap (self-admittedly), not a concrete defect. Let me verify the key facts it hinges on.

The finding's own claims:
- `_FailSeg` is a hand-rolled stand-in (test_worker.py:204-215) ? CONFIRMED by the read.
- Real segmenter Failure returns only verified at segmenter level (test_tracknet_hybrid.py) ? need to check whether any worker-level test drives the real segmenter.
- The finding explicitly states "this is a completeness gap rather than an uncovered hole" and "Both halves are individually locked."

Let me verify there's no existing worker test that flows the real segmenter, and confirm the segmenter registered as `wasb_hybrid` maps to the real class.

6. user (2026-07-06T17:30:50.776Z)

```
deploy\cloudrun\gen_service_yaml.py:36:  * `indexing.default_segmenter=wasb_hybrid` ? the engine
default is yolo_hybrid, so an unpinned
deploy\cloudrun\gen_service_yaml.py:164:        default="wasb_hybrid",
deploy\cloudrun\gen_service_yaml.py:166:        "validated wasb_hybrid.",
scripts\verify_compute_path.py:16:      --segmenter wasb_hybrid --compute cloud --expect
cloud_run --verify-gcp \
tools\generate_highlights.py:47:_WSL_DETECTORS = {"wasb_hybrid", "tracknet_hybrid",
"yolo_hybrid"}
tools\generate_highlights.py:381:        "dumping ~46k PNGs. Only affects wasb_hybrid.",
tests\test_api_cloud_dispatch.py:152:        m.ProcessRequest(video_path="gs://b/clip.mp4",
segmenter="wasb_hybrid")
tests\test_api_cloud_dispatch.py:171:        m.ProcessRequest(video_path="gs://b/clip.mp4",
segmenter="wasb_hybrid")
tests\test_api_cloud_dispatch.py:178:    assert argv[argv.index("--segmenter") + 1] ==
"wasb_hybrid"
tests\test_api_cloud_dispatch.py:197:        m.ProcessRequest(video_path="gs://b/clip.mp4",
segmenter="wasb_hybrid")
tests\test_api_cloud_dispatch.py:210:        m.ProcessRequest(video_path="gs://b/clip.mp4",
segmenter="wasb_hybrid")
tests\test_api_cloud_dispatch.py:229:        m.ProcessRequest(video_path=local_path,
segmenter="wasb_hybrid")
tests\test_api_cloud_dispatch.py:255:        m.ProcessRequest(video_path=local_path,
segmenter="wasb_hybrid")
tests\test_api_cloud_dispatch.py:300:        m.ProcessRequest(video_path=local_path,
segmenter="wasb_hybrid")
tests\test_api_cloud_dispatch.py:324:        m.ProcessRequest(video_path=str(tmp_path /
"up.mp4"), segmenter="wasb_hybrid")
tests\test_api_cloud_dispatch.py:394:        m.ProcessRequest(video_path="gs://b/clip.mp4",
segmenter="wasb_hybrid")
tests\test_api_cloud_dispatch.py:408:            m.ProcessRequest(video_path="gs://b/clip.mp4",
segmenter="wasb_hybrid")
tests\test_api_cloud_dispatch.py:495:            video_path="gs://b/clip.mp4",
segmenter="wasb_hybrid", compute="cloud"
tests\test_api_cloud_dispatch.py:526:            video_path=str(video), segmenter="wasb_hybrid",
compute="cloud"
tests\test_api_cloud_dispatch.py:544:                video_path="gs://b/clip.mp4",
segmenter="wasb_hybrid", compute="banana"
tests\test_api_cloud_dispatch.py:588:        json={"video_path": "gs://b/clip.mp4", "segmenter":
"wasb_hybrid"},
tests\test_api_cloud_dispatch.py:672:            "segmenter": "wasb_hybrid",
tests\test_api_cloud_dispatch.py:699:            video_path="gs://b/clip.mp4",
segmenter="wasb_hybrid", compute="cloud"
tests\test_api_cloud_dispatch.py:773:            video_path=str(video), segmenter="wasb_hybrid",
compute="cloud"
tests\test_api_cloud_dispatch.py:816:            "segmenter": "wasb_hybrid",
tests\test_api_cloud_dispatch.py:870:    early = _wait_progress_message("wasb_hybrid", 12.0)
tests\test_api_cloud_dispatch.py:871:    assert "wasb_hybrid" in early and "starting up" in
early
```

```
tests\test_api_cloud_dispatch.py:872:    later = _wait_progress_message("wasb_hybrid", 125.0)
backend\config\models.py:263:    # (wasb_hybrid / tracknet_hybrid / fusion_hybrid) skip the
Phase-3 AI handoff entirely and
backend\worker\__main__.py:4:        [--segmenter wasb_hybrid] [--config config.json] [--set k=v
...] [--scratch DIR]
backend\worker\__main__.py:479:    from backend.pipeline.segmenters.trajectory_hybrid import
TrajectoryHybridSegmenter
backend\worker\__main__.py:489:    if not isinstance(seg, TrajectoryHybridSegmenter):
backend\worker\__main__.py:492:            f"use a trajectory hybrid (wasb_hybrid /
tracknet_hybrid / fusion_hybrid)."
backend\worker\__main__.py:755:        default="wasb_hybrid",
tests\test_database.py:269:        "vidA", "wasb_hybrid", 2.0, 9.0, stats={"net_crossings": 3}
backend\reporting\harvest.py:23:_AI_SEGMENTERS = {"yolo_hybrid", "tracknet_hybrid",
"wasb_hybrid", "gemini"}
tests\test_deploy_manifests.py:54:    assert cfg["indexing"]["default_segmenter"] ==
"wasb_hybrid"
backend\eval\experiment.py:113:    segmenter: str = "wasb_hybrid"
tests\test_compute_backend.py:185:            video_uri="gs://b/videos/clip.mp4", out_uri=out,
segmenter="wasb_hybrid"
tests\test_compute_backend.py:212:            segmenter="wasb_hybrid",

[Showing results with pagination = limit: 40]
```

7. assistant (2026-07-06T17:30:56.104Z)

Let me verify the cmd_infer unwrap logic and confirm no worker test drives the real segmenter's Failure path end-to-end.

8. user (2026-07-06T17:30:58.251Z)

```
320                 return _fail(2)
321             segmenter_actual = seg_name
322             result = segmenter_cls(db, config).process_video(
323                 video_id, local_video, max_frames=args.max_frames
324             )
325             segmentation_ran = True
326             if isinstance(result, Failure):
327                 print(f"[worker] segmenter FAILED: {result.message}")
328                 return _fail(3)
329             segments = result.value if hasattr(result, "value") else result
330             status = "success"
331             # Loud, never-silent diagnostic: a 0-segment success is almost always a
misconfig, not an
332             # empty match ? the cold-start failure mode (the substrate detector yielded no
usable
333             # trajectories). Explain it so a run is analysable from the log alone (re-run -v
for more).
334             if not segments:
335                 detector_impl = (
336                     os.environ.get("RALLY_DETECTOR_IMPL")
337                     or getattr(config.indexing, "detector_impl", None)
338                     or "auto"
339                 )
340                 logger.warning(
341                     "infer produced 0 segments for video_id=%s (segmenter=%s,
detector_impl=%s). "
342                     "The detector substrate yielded no usable trajectories/windows ? check
the detector "
343                     "logs above; common causes: a missing ${AI_PROVIDER}_API_KEY with "
344                     "skip_ai_handoff=False (the segmenter bails BEFORE detection ? set the
key or "
345                     "indexing.skip_ai_handoff=true; cloud-serving now fails this at
startup), native "
346                     "runner UNAVAILABLE (weights/repo/WASB_PYTHON env), the WASB env
produced no "
```

```
347                     "detections, or the input resolution differs from the tuned proxy
resolution. "
348                     "Re-run with -v for the native command + frame counts.",
349                     video_id,
350                     segmenter_actual,
351                     detector_impl,
352                 )
353         finally:
354             # Best-effort per-video observability report (never raises) ? parity with the
main CLI.
355             emit_indexer_report(
356                 db=db,
357                 video_id=video_id,
358                 video_path=local_video,
359                 config=config,
360                 val_results=val_results,
361                 val_context=val_context,
362                 segmenter_requested=args.segmenter,
363                 segmenter_actual=segmenter_actual,
364                 was_reingest=False,
```

9. user (2026-07-06T17:30:59.738Z)

```
121:    def process_video(self, video_id, path, max_frames=None):
156:    monkeypatch.setattr(wmain.segmenter_registry, "get", lambda name: _FakeSeg)
200:    def process_video(self, video_id, path, max_frames=None):
211:    def process_video(self, video_id, path, max_frames=None):
235:    monkeypatch.setattr(wmain.segmenter_registry, "get", lambda name: _EmptySeg)
282:    monkeypatch.setattr(wmain.segmenter_registry, "get", lambda name: _FailSeg)
```

10. assistant (2026-07-06T17:31:09.575Z)

Verification complete. The facts in the finding are accurate:

1. **`_FailSeg` is a hand-rolled stand-in** (test_worker.py:204-215) ? confirmed. All worker tests monkeypatch `segmenter_registry.get` to return fake segmenter classes (`_FakeSeg`, `_EmptySeg`, `_FailSeg`), never the real `TrajectoryHybridSegmenter`.

2. **No worker test imports/drives the real segmenter** ? the grep shows the only `process_video` occurrences in test_worker.py are on the fake classes, and there is no import of `trajectory_hybrid` / `TrajectoryHybridSegmenter` in the worker tests. The real segmenter's Failure paths are only asserted at the segmenter level in test_tracknet_hybrid.py.

3. **The unwrap point exists as described** (worker/__main__.py:326-329): `if isinstance(result, Failure): ... return _fail(3)`.

However, this is by the finding's own admission a **completeness gap, not a defect**: "Both halves are individually locked, so this is a completeness gap rather than an uncovered hole." There is no concrete input/state that produces wrong behavior in the code as written. The two halves compose correctly today ? the real segmenter returns `Failure`, and cmd_infer branches on `isinstance(result, Failure)`. The failure scenario is explicitly hypothetical ("If a future refactor changed...").

The adversarial-verifier standard is: CONFIRMED only when a concrete input/state produces the claimed wrong behavior in code as written. Here, no wrong behavior exists in the current code ? the finding describes a test-coverage seam that would only bite under a hypothetical future refactor. This is a legitimate low-severity test-completeness observation, not a bug.

11. user (2026-07-06T17:31:19.540Z)

Structured output provided successfully